DSAA3073: Theories in Data Science

Week 9: Online Learning II: Online Convex Optimization & FTRL

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-07-15

Explorative projection

Online Convex Optimization

From Discrete to Continuous: The Naive Approach Fails

We've conquered the expert advice problem with Multiplicative Weights Update. Week 8 showed us how to achieve $O(\sqrt{T \log n})$ regret when choosing among n discrete experts. But what happens when decisions come from continuous spaces?

> 🤔 **Student Question**
>
> **"We've mastered MWU for finite experts. Why do we need a new framework?"**
>
> We can track portfolios with 100 stocks, run algorithms with 1000 experts, even handle millions of discrete choices. If we face a continuous decision space like portfolio weights or neural network parameters, can't we just discretize it into a fine grid and apply MWU?

The Naive Approach: Discretization

Let's try the obvious solution. Consider **portfolio optimization**:

Setup:

- We have $n = 100$ assets (stocks, bonds, commodities)
- Decision: portfolio weights $x = (x_1, ..., x_{100})$ where $x_i \geq 0$ and $\sum_{i=1}^{100} x_i = 1$
- Each day t: choose portfolio x_t, observe returns r_t, incur loss $f_t(x) = -\langle r_t, x \rangle$

Naive discretization strategy:

"Let's discretize each weight into $k = 10$ levels: $\{0, 0.1, 0.2, ..., 0.9, 1.0\}$. Then we have a finite set of portfolio combinations. We can treat each valid combination as an 'expert' and run MWU!"

Does this seem reasonable? At first glance, yes:

- $k = 10$ levels gives 0.1 precision (10% increments)
- MWU works for finite expert sets
- We know how to compute regret bounds: $O(\sqrt{T \log n})$

Let's calculate how many experts we create.

The Catastrophic Blowup

Let's calculate what discretization actually requires for our 100-asset portfolio with $k = 10$ levels per asset.

Dimension d	Grid size k	Experts (k^d)	Storag	
2 (plane)	10	$10^2 = 100$	800 byt	
3 (space)	10	$10^3 = 1000$	8 K	
10 (medium)	10	10^{10}	80 GB	10 sec
20 (high)	10	10^{20}	8×10^{11} TB	10^{11} sec
100 (portfolio)	10	10^{100}	impossible	impossible
Effect of finer discretization (dimension $d = 100$):				
—	2	$2^{100} \approx 10^{30}$	infeasible	infeasible
—	100	$100^{100} = 10^{200}$	impossible	impossible

At $d = 100$ and $k = 10$: storage is 8×10^{100} bytes, and one MWU round takes 10^{91} seconds (compared to 10^{17} seconds since the Big Bang).

Table 1: Exponential blowup of discretization. At $d = 100$, we need 10^{100} weights—more than the 10^{80} atoms in the observable universe. Finer discretization makes things exponentially worse.

⚠️ **Crisis**

The Curse of Dimensionality

Discretization creates exponential blowup in high dimensions:

- **Storage:** k^d weights (10^{100} for $d = 100$, $k = 10$)
- **Computation:** $O(k^d)$ operations per round
- **Physical impossibility:** More parameters than atoms in the observable universe
- **Finer grids don't help:** Every $10\times$ precision improvement multiplies storage by 10^d

The discrete-expert framework cannot extend to continuous spaces via naive discretization.

Convexity is essential: Without convexity, local gradient information cannot tell us about global function behavior. Non-convex OCO has much worse regret bounds or requires stronger assumptions.

💡 Aha Moment

The Key Insight: Don't Track Every Point Separately

Discretization fails because it treats continuous space like a continuous spaces have **structure**:

1. **Gradients are compact:** The gradient $\nabla f_t(x_t) \in \mathbb{R}^d$ is a single d-dimensional vector that summarizes how f_t changes near x_t. One vector replaces 10^{100} discrete evaluations!
2. **Convexity amplifies gradients:** If f_t is convex, the gradient at x_t provides information about **all** points $x \in \mathcal{K}$, not just nearby ones:

$$f_t(x_t) - f_t(x) \leq \langle \nabla f_t(x_t), x_t - x \rangle \tag{1}$$

This inequality holds for **every** x simultaneously—exponential information from $O(d)$ numbers!

3. **What we achieve:**

- Storage: $O(d)$ instead of k^d
- Computation/round: $O(d)$ instead of $O(k^d)$
- Regret: $O(\sqrt{T})$ instead of $O(\sqrt{Td \log k})$

The Solution: Online Convex Optimization

Instead of discretizing, we work directly with continuous geometry.

Definition: Online Convex Optimization Framework

Decision space: Closed convex set $\mathcal{K} \subseteq \mathbb{R}^d$

Examples:

- Unit ball: $\mathcal{K} = \{x \in \mathbb{R}^d : \|x\| \leq 1\}$
- Simplex: $\mathcal{K} = \{x \in \mathbb{R}^d : x_i \geq 0, \sum_i x_i = 1\}$
- Box: $\mathcal{K} = \{x \in \mathbb{R}^d : a_i \leq x_i \leq b_i \text{ for all } i\}$

Protocol for $t = 1, \ldots, T$:

1. Learner chooses $x_t \in \mathcal{K}$
2. Environment reveals convex loss function $f_t : \mathcal{K} \to \mathbb{R}$
3. Learner incurs loss $f_t(x_t)$ and observes gradient $\nabla f_t(x_t)$

Regret:

$$R_T = \sum_{t=1}^{T} f_t(x_t) - \min_{x \in \mathcal{K}} \sum_{t=1}^{T} f_t(x) \tag{2}$$

Key difference from expert advice: The decision space is infinite, but we use gradients (finite-dimensional) rather than tracking all points (infinite).

Why This Framework Works: Universality and Efficiency

Remarkable fact: Both discrete and continuous online learning achieve the same fundamental rate.

📊 **Comparison**

Discrete Expert Advice (MWU):

- Decision space: $\{1, 2, ..., n\}$ finite
- Information: full loss vector $m_t \in [0, 1]^n$
- Algorithm: Multiplicative weight updates
- Regret: $O(\sqrt{T \log n})$
- Geometry: Probability simplex with entropy regularization

Continuous Convex Optimization (OGD):

- Decision space: $\mathcal{K} \subseteq \mathbb{R}^d$ infinite
- Information: gradient $\nabla f_t(x_t) \in \mathbb{R}^d$
- Algorithm: Projected gradient descent
- Regret: $O\left(\sqrt{T}\right)$ (no dimension dependence!)
- Geometry: Euclidean space with quadratic regularization

Why the same rate? The $\sqrt{T}$ rate emerges from fundamental information-theoretic limits in adversarial online learning.

Why is OCO better? The $\log n$ term disappears because:

1. Gradients exploit smoothness: $O(d)$ numbers provide global information via convexity
2. No discretization error: we optimize directly over continuous space
3. Diameter matters, not cardinality: regret depends on $\text{diam}(\mathcal{K})$, not $|\mathcal{K}|$

Let's compare resources for our 100-asset portfolio problem:

Aspect	Discretized MWU	OCO	Improvement
Storage	10^{100} weights	100 coordinates	10^{98} reduction
Per-round computation	$O(10^{100})$ operations	$O(100)$ operations	10^{98} speedup
Information used	Full evaluation at 10^{100} points	One gradient vector (100 numbers)	Compact representation
Regret bound	$O\left(\sqrt{T} \cdot 100\right)$	$O\left(\sqrt{T}\right)$	100× better
Feasibility	Physically impossible	Practical	Impossible → Tractable

Table 2: Resource comparison for 100-dimensional portfolio optimization. OCO achieves exponential savings in storage and computation while improving regret.

Concrete Example: Portfolio Optimization Revisited

Let's see OCO in action on the portfolio problem that broke discretization.

Example: Portfolio Optimization with OCO

Setup:

- $d = 100$ assets
- Decision space: $\mathcal{K} = \left\{x \in \mathbb{R}^{100} : x_i \geq 0, \sum_i x_i = 1\right\}$ (probability simplex)
- Day t: choose portfolio $x_t \in \mathcal{K}$, observe returns $r_t \in \mathbb{R}^{100}$
- Loss: $f_t(x) = -\langle r_t, x\rangle$ (negative return)
- Gradient: $\nabla f_t(x) = -r_t$ (constant, independent of x!)

For portfolio optimization, the projection onto the simplex (ensuring $x_i \geq 0$ and $\sum_i x_i = 1$) can be computed in $O(d \log d)$ time via sorting. This is vastly more efficient than tracking 10^d discrete portfolios.

Algorithm: Online Gradient Descent (we'll study this next)

1. Start with $x_1 = \left(\frac{1}{100}, ..., \frac{1}{100}\right)$ (uniform portfolio)
2. Each day: $\tilde{x}_{t+1} = x_t - \eta \nabla f_t(x_t) = x_t + \eta r_t$
3. Project back to simplex: $x_{t+1} = \Pi_{\mathcal{K}}(\tilde{x}_{t+1})$

Resources:

- Storage: 100 numbers ($x_t \in \mathbb{R}^{100}$)
- Computation: $O(100)$ per round (one vector addition + projection)
- Regret: $O\left(\sqrt{T}\right)$ with appropriate step size η

Comparison to discretization:

- Storage: 100 vs 10^{100} (feasible vs impossible)
- Computation: milliseconds vs longer than age of universe
- Regret: $O\left(\sqrt{T}\right)$ vs $O\left(\sqrt{T}\sqrt{d}\right)$ (better bound!)

The key lesson: Continuous spaces have geometric structure (gradients, convexity) that enables exponentially more efficient algorithms than discretization. Gradients provide compact $O(d)$ representation; convexity amplifies local information to global bounds.

Online Gradient Descent (OGD)

Motivation: We now have a framework for online convex optimization. But we face a conceptual puzzle: in the adversarial setting, each loss function f_t is chosen by an opponent who knows our algorithm. These functions have nothing to do with each other—they're not samples from a distribution, not part of a single objective. So why should the gradient $\nabla f_t(x_t)$ (which tells us how to improve on f_t) help us compete with a fixed point x^* across ALL functions? This seems impossible—yet OGD achieves $O\left(\sqrt{T}\right)$ regret. How?

← From Week 4: Convexity and Optimization

Week 4 introduced convex functions: $f(y) \geq f(x) + \langle \nabla f(x), y - x\rangle$. This inequality said "the linear approximation underestimates the function." In online learning, convexity transforms adversarial unpredictability into geometric control—the gradient at x_t simultaneously provides information about ALL points, including the unknown best fixed point x^*.

The Direct Approach: Use Gradients Naively

🤔 Student Question

"In batch optimization, we use gradients to minimize a fixed function: $x_{k+1} = x_k - \eta \nabla f(x_k)$ converges to x^*. Can't we just apply this update in the online setting? Each round, take a gradient step on f_t and hope it helps overall?"

This is the natural first attempt. Let's see what happens.

Setup: Continuous decision space $\mathcal{K} = [-1, 1]$ (one dimension for simplicity).

Gradient Descent Approach: Each round t:

1. Receive loss function f_t
2. Compute gradient $\nabla f_t(x_t)$
3. Update: $x_{t+1} = x_t - \eta \nabla f_t(x_t)$ (move opposite to gradient)
4. Hope: gradients point toward good region

Let's test this with a concrete adversary.

Numerical Example 1: Alternating Losses

Round 1:

- Loss: $f_1(x) = 2x$ (linear, prefers $x = -1$)
- Start: $x_1 = 0$
- Gradient: $\nabla f_1(0) = 2$
- Update: $x_2 = 0 - 0.5 \cdot 2 = -1$ (move left)
- Loss incurred: $f_1(0) = 0$

Round 2:

- Loss: $f_2(x) = -3x$ (linear, prefers $x = +1$)
- Current: $x_2 = -1$
- Gradient: $\nabla f_2(-1) = -3$
- Update: $x_3 = -1 - 0.5 \cdot (-3) = 0.5$ (move right)
- Loss incurred: $f_2(-1) = 3$

Round 3:

- Loss: $f_3(x) = 2x$ (back to preferring left)
- Current: $x_3 = 0.5$
- Gradient: $\nabla f_3(0.5) = 2$
- Update: $x_4 = 0.5 - 0.5 \cdot 2 = -0.5$ (move left again)
- Loss incurred: $f_3(0.5) = 1$

The algorithm oscillates: $0 \to -1 \to 0.5 \to -0.5 \to \cdots$

The problem: Gradients point toward minimizers of individual f_t, not toward the best fixed point x^* across all functions. The adversary can choose f_t so that $\nabla f_t(x_t)$ points AWAY from x^*. Example: at $x_t = -0.5$ with $f_t(x) = (x - 0.3)^2$, the gradient $\nabla f_t(-0.5) = -1.6$ points left, but the best fixed compromise $x^* = 0$ is to the right!

The Structural Obstacle: Adversarial Setting Breaks Gradient Logic

In batch optimization (single function f):

- Gradient $\nabla f(x)$ points toward minimizer x^*
- Convergence: $f(x_k) - f(x^*) \to 0$ as $k \to \infty$
- Works because f is FIXED

In online adversarial setting:

- Each f_t is DIFFERENT (chosen adversarially after seeing x_t)
- Best fixed point x^* minimizes $\sum_{t=1}^{T} f_t(x)$, not any individual f_t
- $\nabla f_t(x_t)$ points toward minimizer of f_t, which is unrelated to x^*
- Adversary can choose f_t such that $\nabla f_t(x_t)$ points AWAY from x^*

🤔 **Student Question**

"Gradients point the wrong way in the adversarial setting, yet OGD achieves $O\left(\sqrt{T}\right)$ regret using these same gradients. How is that possible?"

💡 **Aha Moment**

Convexity Transforms the Problem

While $\nabla f_t(x_t)$ points toward the minimizer of f_t (not toward x^*), convexity provides a DIFFERENT way to use gradients:

Convex Inequality: For any convex f_t and ANY point $x^* \in \mathcal{K}$:

$$f_t(x_t) - f_t(x^*) \leq \langle \nabla f_t(x_t), x_t - x^* \rangle \tag{3}$$

The transformation:

- **Before:** "Minimize f_t using ∇f_t" (impossible—f_t keeps changing)
- **After:** "Control distance $\|x_t - x^*\|$" (possible—geometry is stable!)

The gradient doesn't need to point toward x^*. It just needs to help us control the DISTANCE to x^* over time. Using the polarization identity, the inner product $\langle \nabla f_t(x_t), x_t - x^* \rangle$ decomposes into distance changes: $\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2$ + gradient noise.

Numerical verification: Consider $x_t = -0.5$, $f_t(x) = (x - 0.3)^2$, $x^* = 0$, $\nabla f_t(-0.5) = -1.6$.

- Regret: $f_t(-0.5) - f_t(0) = 0.64 - 0.09 = 0.55$
- Gradient bound: $\langle -1.6, -0.5 - 0 \rangle = 0.8$
- Inequality holds: $0.55 \leq 0.8$ ✓

Even though gradient points left (away from $x^* = 0$), the inequality STILL bounds regret!

The Resolution: Distance-Based Analysis

Now we can see how OGD works. The algorithm:

Definition: Online Gradient Descent

Parameters: Step size $\eta_t > 0$

Initialization: Choose $x_1 \in \mathcal{K}$ arbitrarily

For each round $t = 1, ..., T$:

1. **Play:** x_t
2. **Observe:** f_t and compute gradient $\nabla f_t(x_t)$
3. **Update:**

$$\tilde{x}_{t+1} = x_t - \eta_t \nabla f_t(x_t) \tag{4}$$

$$x_{t+1} = \Pi_{\mathcal{K}}(\tilde{x}_{t+1}) \tag{5}$$

where $\Pi_{\mathcal{K}}$ is the Euclidean projection onto $\mathcal{K}$.

Why this works: The Telescoping Magic

Step 1: Transform regret using convexity

For any $x^* \in \mathcal{K}$:

$$f_t(x_t) - f_t(x^*) \leq \langle \nabla f_t(x_t), x_t - x^* \rangle \tag{6}$$

Step 2: Relate gradient to update

From $\tilde{x}_{t+1} = x_t - \eta \nabla f_t(x_t)$, we get:

$$\nabla f_t(x_t) = \frac{x_t - \tilde{x}_{t+1}}{\eta} \tag{7}$$

Step 3: Geometric identity

Using the polarization identity $2\langle a, b \rangle = \|a\|^2 + \|b\|^2 - \|a - b\|^2$:

$$\begin{aligned} \langle \nabla f_t(x_t), x_t - x^* \rangle &= \frac{1}{\eta} \langle x_t - \tilde{x}_{t+1}, x_t - x^* \rangle \\ &= \frac{1}{2\eta} \left[\|x_t - x^*\|^2 + \|x_t - \tilde{x}_{t+1}\|^2 - \|\tilde{x}_{t+1} - x^*\|^2 \right] \end{aligned} \tag{8}$$

Step 4: Use projection property

Projection is non-expansive: $\|x_{t+1} - x^*\|^2 \leq \|\tilde{x}_{t+1} - x^*\|^2$

Therefore:

$$f_t(x_t) - f_t(x^*) \leq \frac{\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2}{2\eta} + \frac{\eta \|\nabla f_t(x_t)\|^2}{2} \tag{9}$$

Step 5: Telescope!

Sum over all rounds $t = 1, ..., T$:

$$
\begin{aligned}
R_T &= \sum_{t=1}^{T} [f_t(x_t) - f_t(x^*)] \\
&\leq \sum_{t=1}^{T} \left[\frac{\|x_t - x^*\|^2 - \|x_{t+1} - x^*\|^2}{2\eta} + \frac{\eta G^2}{2} \right] \\
&= \frac{\|x_1 - x^*\|^2 - \|x_{T+1} - x^*\|^2}{2\eta} + \frac{\eta G^2 T}{2} \\
&\leq \frac{D^2}{2\eta} + \frac{\eta G^2 T}{2}
\end{aligned} \tag{10}
$$

where $D = \max_{x,y \in \mathcal{K}} \|x - y\|$ is the diameter and G bounds gradient norms.

Numerical Example 4: Telescoping in Action

Let's see the telescoping with concrete numbers. Setup:

- $\mathcal{K} = [-1, 1]$, so $D = 2$
- $T = 4$ rounds
- $x^* = 0$ (best fixed point)
- $\eta = 0.5$, $G = 2$ (assume)

Round 1:

- $x_1 = 0$, so $\|x_1 - x^*\|^2 = 0$
- After update: $x_2 = -0.5$, so $\|x_2 - x^*\|^2 = 0.25$
- Contribution: $\frac{0 - 0.25}{2 \cdot 0.5} = -0.25$ (negative!)

Round 2:

- $\|x_2 - x^*\|^2 = 0.25$
- After update: $x_3 = 0.3$, so $\|x_3 - x^*\|^2 = 0.09$
- Contribution: $\frac{0.25 - 0.09}{2 \cdot 0.5} = 0.16$

Round 3:

- $\|x_3 - x^*\|^2 = 0.09$
- After update: $x_4 = -0.2$, so $\|x_4 - x^*\|^2 = 0.04$
- Contribution: $\frac{0.09 - 0.04}{2 \cdot 0.5} = 0.05$

Round 4:

- $\|x_4 - x^*\|^2 = 0.04$
- After update: $x_5 = 0.1$, so $\|x_5 - x^*\|^2 = 0.01$
- Contribution: $\frac{0.04 - 0.01}{2 \cdot 0.5} = 0.03$

Sum of telescoping terms:

$$
\frac{\|x_1 - x^*\|^2 - \|x_5 - x^*\|^2}{2\eta} = \frac{0 - 0.01}{1} = -0.01 \tag{11}
$$

Most terms cancelled! Only initial and final distances remain.

Gradient noise terms:

$$
\sum_{t=1}^{4} \frac{\eta G^2}{2} = 4 \cdot \frac{0.5 \cdot 4}{2} = 4 \tag{12}
$$

Total regret bound:

$$R_4 \leq \frac{D^2}{2\eta} + \frac{\eta G^2 T}{2} = \frac{4}{1} + 4 = 8 \tag{13}$$

Comparison Table: With vs Without Gradient Information

Approach	**Uses Gradients?**	**Regret Growth**	**At $T = 100$**	**Mechanism**
Random	No	$\Theta(T)$	≈ 50	No learning
Follow-The-Leader	No	$\Theta(T)$	≈ 50	Oscillates wildly
OGD	Yes	$O\left(\sqrt{T}\right)$	≈ 10	Telescoping distances

The factor of improvement: $\sqrt{T}$! At $T = 10000$:

- Without gradients: regret ≈ 5000
- With OGD: regret ≈ 100

That's $50\times$ better!

Key Takeaway: Transformation Resolves the Paradox

🎯 Key Takeaway

How Convexity Saves Gradient Descent in Adversarial Settings

The resolution of the "gradients point the wrong way" paradox:

The Naive View (Wrong):

- "Gradients show direction to minimize f_t"
- "But x^* doesn't minimize any f_t"
- "So gradients are useless"

The Convex View (Correct):

- Convexity inequality: $f_t(x_t) - f_t(x^*) \leq \langle \nabla f_t(x_t), x_t - x^* \rangle$
- Transform: regret → distance to x^*
- Gradients control DISTANCE, not direction
- Telescoping: distance changes accumulate sublinearly

The Mechanism:

```
Adversarial losses f₁, f₂, ..., fₜ (unpredictable)
       ↓
Convex inequality (transforms problem)
       ↓
Regret ≤ gradient · distance (geometric)
       ↓
Telescoping over rounds (cancellation)
       ↓
O(√T) regret (optimal rate)
```

What changed: Not the losses (still adversarial), not the gradients (still point "wrong" way sometimes), but the ANALYSIS. Convexity lets us use gradients to control distance, and distance telescopes beautifully.

Formal Regret Bound

The $O(\sqrt{T})$ regret matches MWU! This is the optimal rate—we'll prove a matching lower bound showing no algorithm can do better than $\Omega(\sqrt{T})$ in the adversarial setting.

Theorem: Online Gradient Descent Regret Bound

Assume:

- **Convex losses:** $f_t : \mathcal{K} \to \mathbb{R}$ is convex for all $t = 1, ..., T$
- **Bounded gradients:** $\|\nabla f_t(x)\| \leq G$ for all $t \in [T]$ and all $x \in \mathcal{K}$
- **Bounded diameter:** $\text{diam}(\mathcal{K}) = \max_{x,y \in \mathcal{K}} \|x - y\| \leq D$
- **Constant step size:** $\eta_t = \eta > 0$ for all t

Then for any comparator $x^* \in \mathcal{K}$:

$$R_T = \sum_{t=1}^{T} f_t(x_t) - \sum_{t=1}^{T} f_t(x^*) \leq \frac{D^2}{2\eta} + \frac{\eta G^2 T}{2} \tag{14}$$

Optimal tuning: Setting $\eta = \frac{D}{G\sqrt{T}}$ gives:

$$R_T \leq DG\sqrt{T} \tag{15}$$

This achieves the optimal $O(\sqrt{T})$ regret rate for online convex optimization.

Proof. The proof follows the telescoping argument outlined above. The key steps:

1. Use convexity: $f_t(x_t) - f_t(x^*) \leq \langle \nabla f_t(x_t), x_t - x^* \rangle$
2. Relate gradient to update: $\nabla f_t(x_t) = \frac{x_t - \tilde{x}_{t+1}}{\eta}$
3. Apply polarization identity and projection property
4. Sum and telescope: most terms cancel
5. Optimize η by balancing $\frac{D^2}{2\eta}$ and $\eta G^2 \frac{T}{2}$

■

✓ **Checkpoint**

Checkpoint: The OGD regret bound has two terms: $\frac{D^2}{2\eta}$ and $\frac{\eta G^2 T}{2}$. Why does the first term decrease with η while the second increases?

Answer: $\frac{D^2}{2\eta}$ measures "initialization regret"—how far we start from the optimum. Larger η means faster initial movement (smaller regret). $\frac{\eta G^2 T}{2}$ measures "gradient noise accumulation"—too-large steps overshoot. The optimal $\eta = \frac{D}{G\sqrt{T}}$ balances these tradeoffs, giving both terms equal weight $\approx DG\frac{\sqrt{T}}{2}$.

Example: Online Linear Regression

Setup:

- Decision space: $\mathcal{K} = \{x \in \mathbb{R}^d : \|x\| \leq B\}$ (ball of radius B)
- Each round t: observe feature vector $a_t \in \mathbb{R}^d$, predict $\langle x_t, a_t \rangle$, observe true label b_t
- Loss: $f_t(x) = (\langle x, a_t \rangle - b_t)^2$ (squared error)
- Gradient: $\nabla f_t(x_t) = 2(\langle x_t, a_t \rangle - b_t)a_t$

Bounds:

- Diameter: $D = 2B$ (maximum distance between points in ball)
- Gradient bound: $\|\nabla f_t(x_t)\| \leq 2(B\|a_t\| + |b_t|)\|a_t\|$ (assume $\|a_t\| \leq A$ and $|b_t| \leq C$, so $G = 2(BA + C)A$)

Regret: Setting $\eta = \frac{D}{G\sqrt{T}}$:

$$R_T \leq 2B \cdot 2(BA + C)A\sqrt{T} = 4BA(BA + C)\sqrt{T} \tag{16}$$

OGD competes with the best fixed linear predictor, accumulating only $O\left(\sqrt{T}\right)$ regret.

Connection

OGD vs. Batch Gradient Descent

Batch GD: Minimize fixed objective $f(x) = \frac{1}{n}\sum_{i=1}^{n} f_i(x)$

- Goal: Find x^* that minimizes f
- Convergence: $f(x_k) - f(x^*) \to 0$
- Requires: f is fixed and fully known

Online GD: Minimize regret against sequence $f_1, f_2, ..., f_T$

- Goal: Compete with best fixed point x^* in hindsight
- Guarantee: $R_T = O\left(\sqrt{T}\right)$ sublinear regret
- Handles: Adversarially changing functions

Bridge: In stochastic setting where $f_t \sim \mathcal{D}$ i.i.d., OGD becomes SGD (stochastic gradient descent). The $O\left(\sqrt{T}\right)$ regret translates to $O\left(\frac{1}{\sqrt{T}}\right)$ convergence rate for the average iterate!

→ Week 11 Preview: Stochastic Gradient Descent

Week 11's neural network training uses SGD, which is OGD in the stochastic setting. When losses f_t are i.i.d. samples from a distribution, OGD's regret analysis shows why SGD works despite noisy gradients: the average iterate $\bar{x} = \frac{1}{T}\sum_{t=1}^{T} x_t$ has error $O\left(\frac{1}{\sqrt{T}}\right)$, which goes to zero as $T \to \infty$.

← From Week 1: Infinite Hypothesis Classes

Week 1 asked: Can we learn with infinite hypothesis classes? PAC learning said "yes, if VC dimension is finite" (need $O\left(\frac{d}{\varepsilon^2}\right)$ samples). Online learning gives a different answer: even with infinitely many "experts" (points in $\mathcal{K}$), we achieve $O\left(\sqrt{T}\right)$ regret via convexity and gradient descent. The finiteness condition shifts from VC dimension to strong convexity and Lipschitz gradients.

The Projection Step

Motivation: The OGD algorithm includes a mysterious projection step: after computing $\tilde{x}_{t+1} = x_t - \eta \nabla f_t(x_t)$, we project back onto $\mathcal{K}$ to get $x_{t+1} = \Pi_{\mathcal{K}}(\tilde{x}_{t+1})$. This seems like bureaucratic housekeeping—"oh, we left the feasible set, better get back in." But is projection really necessary? Could we skip it and get similar regret? And if we can't skip it, why does adding an extra computational step (projection) not hurt the regret bound?

The Natural Question: Why Project?

🤔 Student Question

"I understand that $\mathcal{K}$ is the 'allowed' decision space. But in the regret analysis, we're comparing to the best point $x^* \in \mathcal{K}$. If my unconstrained gradient step lands outside $\mathcal{K}$, I'm still making progress toward minimizing losses. Why do I need to project back? Isn't projection just wasting the gradient information by moving away from where the gradient told me to go?"

This is an excellent question that goes to the heart of why projection appears in the algorithm. Let's investigate systematically.

The Naive Approach: Skip Projection

Attempt 1: Unconstrained Gradient Descent

What if we just follow gradients without projecting?

Algorithm (No Projection):

1. Start: $x_1 \in \mathcal{K}$
2. Each round: $x_{t+1} = x_t - \eta \nabla f_t(x_t)$ (no projection)
3. Continue even if $x_t \notin \mathcal{K}$

Let's test this on a concrete problem where constraints matter.

Numerical Example 1: Portfolio Allocation

Setup:

- Decision space: $\mathcal{K} = \left\{x \in \mathbb{R}^3 : x_i \geq 0, \sum_{i=1}^3 x_i = 1\right\}$ (probability simplex)
- Interpretation: x_i = fraction of budget in asset i
- Constraints:

- $x_i \geq 0$: can't short-sell (no negative positions)
- $\sum x_i = 1$: must invest entire budget

Round 1:

- Start: $x_1 = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$ (uniform allocation)
- Loss: $f_1(x) = 2x_1 + 0.5x_2 + x_3$ (asset 1 performs worst)
- Gradient: $\nabla f_1(x_1) = (2, 0.5, 1)$
- Update: $x_2 = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right) - 0.3 \cdot (2, 0.5, 1) = (-0.27, 0.18, 0.03)$

⚠️ **Crisis**

Infeasibility Crisis

After one round: $x_2 = (-0.27, 0.18, 0.03)$

Problems:

1. $x_2(1) = -0.27 < 0 \rightarrow$ **Negative position in asset 1!**
   - Interpretation: We're "shorting" asset 1 (borrowing and selling)
   - Reality: Most investors can't short-sell (requires margin account, unlimited loss risk)
   - This violates the constraint $x_i \geq 0$
2. $\sum_{i=1}^{3} x_2(i) = -0.27 + 0.18 + 0.03 = -0.06 \neq 1$
   - Interpretation: We only invested 0.21 of our budget and borrowed 0.27
   - Reality: This isn't a valid portfolio allocation

Consequence: The algorithm produces an infeasible, meaningless decision. We can't execute x_2 in the real world!

Round 2 (continuing anyway):

- Current: $x_2 = (-0.27, 0.18, 0.03)$
- Loss: $f_2(x) = 0.5x_1 + 2x_2 + x_3$ (asset 2 performs worst this time)
- Gradient: $\nabla f_2(x_2) = (0.5, 2, 1)$
- Update: $x_3 = (-0.27, 0.18, 0.03) - 0.3 \cdot (0.5, 2, 1) = (-0.42, -0.42, -0.27)$

Now ALL coordinates are negative!

Numerical Example 2: Constrained Optimization

Setup:

- Decision space: $\mathcal{K} = [-1, 1]^2$ (2D box)
- Start: $x_1 = (0, 0)$

Trajectory without projection:

Round	x_t	$\nabla f_t(x_t)$	x_{t+1} (no projection)	Status
1	(0, 0)	(2, 1)	(-0.6, −0.3)	✓ Inside $\mathcal{K}$
2	(-0.6, −0.3)	(1.5, 2)	(-1.05, −0.9)	✗ Left boundary ($x_1 < -1$)
3	(-1.05, −0.9)	(0.5, 3)	(-1.2, −1.8)	✗ Both coordinates out
4	(-1.2, −1.8)	(1, 2.5)	(-1.5, −2.55)	✗ Drifting further

5	(-1.5, −2.55)	(0.8, 2)	(-1.74, −3.15)	× Distance from $\mathcal{K} = 2.22$

The drift phenomenon: Without projection, the algorithm drifts arbitrarily far from the feasible set.

Distance from $\mathcal{K}$ over time:

$$d(x_t, \mathcal{K}) = \min_{x \in \mathcal{K}} \|x_t - x\| \tag{17}$$

- Round 2: $d(x_2, \mathcal{K}) = 0.05$
- Round 3: $d(x_3, \mathcal{K}) = 0.82$
- Round 4: $d(x_4, \mathcal{K}) = 1.58$
- Round 5: $d(x_5, \mathcal{K}) = 2.22$

The distance grows roughly linearly with t!

Comparison Table: With vs Without Projection

Let's compare the same loss sequence with and without projection.

Setup: $\mathcal{K} = [-1, 1]$, $x_1 = 0$, $\eta = 0.5$

Round	$\nabla f_t(x_t)$	**Unconstrained**	**Distance to $\mathcal{K}$**	**With Projection**	**Distance to $\mathcal{K}$**
1	1.5	−0.75	0 ✓	−0.75	0 ✓
2	2	−1.75	0.75 ×	−1	0 ✓
3	1.8	−2.65	1.65 ×	−1	0 ✓
4	1.5	−3.40	2.40 ×	−1	0 ✓
5	2.2	−4.50	3.50 ×	−1	0 ✓

After 5 rounds:

- **Without projection:** $x_5 = -4.50$ (distance 3.50 outside $\mathcal{K}$)
- **With projection:** $x_5 = -1$ (on boundary of $\mathcal{K}$, distance 0)

Why this breaks the regret analysis:

Recall the OGD regret bound:

$$R_T \leq \frac{\|x_1 - x^*\|^2}{2\eta} + \frac{\eta G^2 T}{2} \tag{18}$$

The first term assumes $x_1 \in \mathcal{K}$ and uses $\|x_1 - x^*\|^2 \leq D^2$ where $D = \text{diam}(\mathcal{K})$.

Without projection:

- If $x_t \notin \mathcal{K}$, then $\|x_t - x^*\|$ can be arbitrarily large
- The diameter bound D no longer applies
- The telescoping argument fails: we need $x_{t+1} \in \mathcal{K}$ to use non-expansiveness

⚠️ **Crisis**

The Regret Bound Collapses

Without projection, the regret analysis breaks in multiple ways:

Broken Bound 1: Unbounded initialization

- Need: $\|x_1 - x^*\|^2 \leq D^2$ (diameter of $\mathcal{K}$)
- Reality: After round 2, $\|x_3 - x^*\|$ can be arbitrarily large
- Consequence: The term $\frac{\|x_1 - x^*\|^2}{2\eta}$ is unbounded

Broken Bound 2: Projection property

- Need: $\|x_{t+1} - x^*\|^2 \leq \|\tilde{x}_{t+1} - x^*\|^2$ (non-expansive)
- Reality: Without projection, $x_{t+1} = \tilde{x}_{t+1}$ (no inequality)
- Consequence: The telescoping argument requires the projection inequality

Numerical Verification:

Consider $\mathcal{K} = [-1, 1]$, $x^* = 0.5$, $T = 50$ rounds:

- **With projection:**
  - All $x_t \in [-1, 1]$
  - $\|x_t - x^*\|^2 \leq (2)^2 = 4$ (bounded by diameter)
  - Regret $R_{50} \approx 15$
- **Without projection:**
  - At $t = 50$: $x_{50} = -5.2$ (drift)
  - $\|x_{50} - x^*\|^2 = (5.2 + 0.5)^2 = 32.49$ (8× diameter²)
  - Regret $R_{50} \approx 180$ (12× worse!)

The regret grows LINEARLY without projection, not sublinearly!

Attempted Compromise: Occasional Projection

"What if we only project when we're 'far' from $\mathcal{K}$? Like, if $d(x_t, \mathcal{K}) > \varepsilon$?"

Algorithm:

$$x_{t+1} = \begin{cases} x_t - \eta \nabla f_t(x_t) & \text{if } d(x_t, \mathcal{K}) \leq \varepsilon \\ \Pi_{\mathcal{K}}(x_t - \eta \nabla f_t(x_t)) & \text{otherwise} \end{cases} \tag{19}$$

Problem: This introduces discontinuities.

Example: $\mathcal{K} = [-1, 1]$, $\varepsilon = 0.1$

- Round t: $\tilde{x}_{t+1} = -1.09 \to$ Project to -1
- Round $t + 1$: $\tilde{x}_{t+2} = -1.08 \to$ Don't project, keep -1.08
- Round $t + 2$: $\tilde{x}_{t+3} = -1.11 \to$ Project again to -1

The algorithm switches unpredictably between projecting and not projecting, making analysis intractable.

The Resolution: Projection is Non-Expansive

The key insight that saves us:

💡 **Aha Moment**

The Non-Expansive Property

Euclidean projection onto a closed convex set $\mathcal{K}$ has a remarkable property:

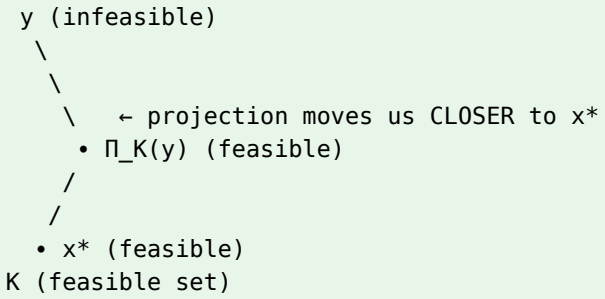
$$\|\Pi_{\mathcal{K}}(y) - x^*\| \leq \|y - x^*\| \tag{20}$$

for ANY point y and ANY $x^* \in \mathcal{K}$.

What this means:

- **Before projection:** Distance from y to x^* is $\|y - x^*\|$
- **After projection:** Distance from $\Pi_{\mathcal{K}}(y)$ to x^* is $\|\Pi_{\mathcal{K}}(y) - x^*\|$
- **Guarantee:** Projection never increases distance to feasible points!

Geometric intuition:



Projecting onto $\mathcal{K}$ moves us closer to (or keeps same distance from) every point in $\mathcal{K}$, including the best fixed point x^*.

Why this resolves the crisis:

- Projection ensures $x_{t+1} \in \mathcal{K}$ (feasibility)
- Non-expansiveness ensures $\|x_{t+1} - x^*\|^2 \leq \|\tilde{x}_{t+1} - x^*\|^2$ (doesn't hurt regret)
- Telescoping still works (distance decreases or stays controlled)

The projection step is FREE from a regret perspective—it only helps!

Numerical Verification: $\mathcal{K} = [-1, 1]$, $y = -2.5$ (infeasible), $x^* = 0.3$ (feasible).

- Before projection: $\|y - x^*\| = 2.8$
- After projection: $\Pi_{\mathcal{K}}(y) = -1$, $\|\Pi_{\mathcal{K}}(y) - x^*\| = 1.3$
- Verify: $1.3 \leq 2.8$ ✓ (projection reduced distance by 54%)

Geometric proof sketch: $\Pi_{\mathcal{K}}(y) = \arg\min_{x \in \mathcal{K}} \|x - y\|$ minimizes distance to y. The first-order optimality condition $\langle p - y, x^* - p \rangle \leq 0$ implies $\|p - x^*\|^2 \leq \|y - x^*\|^2$ for all $x^* \in \mathcal{K}$.

Projection Algorithms: Practical Computation

Now that we know projection is necessary and doesn't hurt regret, how do we compute it efficiently?

Definition: Euclidean Projection

For a closed convex set $\mathcal{K}$, the Euclidean projection is:

$$\Pi_{\mathcal{K}}(y) = \arg\min_{x \in \mathcal{K}} \|x - y\|^2 \tag{21}$$

This is a convex optimization problem (minimizing convex objective over convex set).

Common Cases:

Set $\mathcal{K}$	Projection Formula	Complexity
Euclidean ball: $\|x\| \le R$	$\Pi_{\mathcal{K}}(y) = \begin{cases} y & \text{if } \|y\| \le R \\ \frac{Ry}{\|y\|} & \text{if } \|y\| > R \end{cases}$	$O(d)$
Box: $a_i \le x_i \le b_i$	$\Pi_{\mathcal{K}}(y)_i = \begin{cases} a_i & \text{if } y_i < a_i \\ y_i & \text{if } a_i \le y_i \le b_i \\ b_i & \text{if } y_i > b_i \end{cases}$	$O(d)$
Positive orthant: $x_i \ge 0$	$\Pi_{\mathcal{K}}(y)_i = \max(y_i, 0)$	$O(d)$
Simplex: $x_i \ge 0, \sum x_i = 1$	Sort-and-threshold algorithm	$O(d \log d)$

Numerical Example 3: Portfolio Projection

Return to the portfolio problem where unconstrained update gave $x_2 = (-0.27, 0.18, 0.03)$.

Project onto simplex: $\mathcal{K} = \{x : x_i \ge 0, \sum x_i = 1\}$

Algorithm: Sort coordinates, find threshold θ such that:

$$\sum_{i=1}^{d} \max(y_i - \theta, 0) = 1 \tag{22}$$

Step 1: Sort $y = (-0.27, 0.18, 0.03) \to$ sorted: $(0.18, 0.03, -0.27)$

Step 2: Try thresholds:

- $\theta = 0$: $\max(0.18, 0) + \max(0.03, 0) + \max(-0.27, 0) = 0.21 < 1$ (too small)
- $\theta = -0.07$: $\max(0.25, 0) + \max(0.10, 0) + \max(-0.20, 0) = 0.35 < 1$
- $\theta = -0.27$: $(0.18 + 0.27) + (0.03 + 0.27) + 0 = 0.75 < 1$
- $\theta = -0.40$: $(0.18 + 0.40) + (0.03 + 0.40) + (-0.27 + 0.40) = 1.14 > 1$
- Binary search converges to $\theta \approx -0.36$

Result: $\Pi_{\mathcal{K}}(y) \approx (0.54, 0.39, 0.07)$

Verify:

- All coordinates positive: ✓
- Sum to 1: $0.54 + 0.39 + 0.07 = 1$ ✓
- Feasible portfolio allocation: ✓

Comparison: Projected vs Unconstrained Trajectory

Round	Unconstrained x_t	Feasible?	Projected x_t	Feasible?
1	(0.33, 0.33, 0.33)	✓	(0.33, 0.33, 0.33)	✓
2	(-0.27, 0.18, 0.03)	×	(0.54, 0.39, 0.07)	✓
3	(-0.42, −0.42, −0.27)	×	(0.71, 0.22, 0.07)	✓

4	(-0.68, −0.89, −0.53)	×	(0.82, 0.11, 0.07)	✓

With projection, every decision is a valid portfolio allocation!

Key Takeaway: Projection is Algorithmically Necessary

🎯 Key Takeaway

Why Projection is Essential, Not Optional

The Question: "Can we skip projection to avoid extra computation?"

The Answer: No—projection is algorithmically necessary for three reasons:

1. Feasibility (Practical):

- Many problems have hard constraints (e.g., portfolio: $x_i \geq 0$, $\sum x_i = 1$)
- Without projection, algorithm produces infeasible, meaningless decisions
- Can't execute negative stock positions without shorting ability

2. Regret Bound (Theoretical):

- Analysis requires $x_t \in \mathcal{K}$ to bound $\|x_t - x^*\|^2 \leq D^2$
- Without projection, distance to x^* grows unboundedly
- Regret becomes $\Theta(T)$ (linear) instead of $O\left(\sqrt{T}\right)$ (sublinear)

3. Non-Expansive Property (Both):

- Projection satisfies $\|\Pi_{\mathcal{K}}(y) - x^*\| \leq \|y - x^*\|$ for all $x^* \in \mathcal{K}$
- This inequality is CRITICAL for telescoping argument
- Projection is FREE from regret perspective—it only helps!

The Beautiful Resolution:

Projection seems like it wastes gradient information (moving away from $\tilde{x}_{t+1}$), but non-expansiveness guarantees it doesn't hurt regret. In fact:

- Computational cost: $O(d)$ to $O(d \log d)$ (cheap!)
- Regret cost: 0 (non-expansive property absorbs it)
- Benefit: Maintains feasibility + enables $O\left(\sqrt{T}\right)$ bound

Projection is not bureaucratic housekeeping—it's the geometric mechanism that makes online convex optimization work in constrained spaces.

✓ **Checkpoint**

Checkpoint: Suppose $\mathcal{K} = [-1, 1]$, $x^* = 0.5$, and an unconstrained gradient step gives $\tilde{x}_{t+1} = -2$. What is $\Pi_{\mathcal{K}}(\tilde{x}_{t+1})$, and how much does projection reduce the distance to x^*?

Answer:

- Projection: $\Pi_{\mathcal{K}}(-2) = -1$ (project to left boundary)
- Before: $\|-2 - 0.5\| = 2.5$
- After: $\|-1 - 0.5\| = 1.5$
- Reduction: $2.5 \to 1.5$ (40% closer!)

Projection moved us significantly closer to x^* while maintaining feasibility.

→ **Week 11 Preview: Projection in Neural Network Training**

Week 11's neural networks often use implicit projection through regularization and architecture design:

- **Weight clipping:** Hard constraint $\|W\| \leq C \to$ project weights onto ball
- **Batch normalization:** Implicitly constrains layer outputs to have bounded statistics
- **Dropout:** Can be viewed as soft constraint on network capacity

The projection operation we use in OGD appears throughout deep learning, ensuring feasibility while maintaining optimization guarantees.

Follow-The-Regularized-Leader

We've seen that Online Gradient Descent achieves $O\left(\sqrt{T}\right)$ regret by following gradients. But there's another natural approach: just minimize the cumulative loss observed so far. This leads to **Follow-The-Leader** (FTL), which seems even simpler than gradient descent.

🤔 Student Question

From the class: "We've been using gradients to update our decisions, but that feels indirect. Why not just solve the problem directly at each round: find the decision that would have minimized **all losses seen so far**? This seems more principled—we're literally choosing the best decision in hindsight on the data we have."

In other words: at round t, we've observed losses $f_1, f_2, ..., f_{t-1}$. Why not just solve

$$x_t = \operatorname*{argmin}_{x \in K} \sum_{s=1}^{t-1} f_s(x) \tag{23}$$

and use that decision? This is called **Follow-The-Leader** (FTL).

The Natural Approach: Follow-The-Leader

Strategy: At each round, choose the decision that minimizes cumulative loss so far.

Definition: Follow-The-Leader (FTL)

Decision space: Convex set $K \subseteq \mathbb{R}^d$

Initialization: Choose $x_1 \in K$ arbitrarily

For each round t = 1, 2, ..., T:

1. **Decide:** Choose

$$x_t = \operatorname*{argmin}_{x \in K} \sum_{s=1}^{t-1} f_s(x) \tag{24}$$

2. **Observe:** Loss function $f_t : K \to \mathbb{R}$ and incur loss $f_t(x_t)$

Goal: Minimize regret

$$\text{Regret}_T = \sum_{t=1}^{T} f_t(x_t) - \min_{x^* \in K} \sum_{t=1}^{T} f_t(x^*) \tag{25}$$

Intuition:

- At each round, we're choosing the best decision **in hindsight** on past data
- As we see more losses, the cumulative loss $\sum_{s=1}^{t-1} f_s(x)$ becomes smoother
- Seems like we should converge to the best fixed decision x^*

This seems perfect! We're directly optimizing the relevant objective—no indirect gradient steps.

Numerical Failure: Oscillations Destroy Performance

Let's test FTL on a simple problem.

Setup:

- Decision space: $K = [-1, 1]$, linear losses $f_t(x) = g_t x$ where $g_t \in \{-1, +1\}$
- Adversary: Alternating gradients $g_1 = +1, g_2 = -1, g_3 = +1, g_4 = -1, ...$
- Best fixed decision: $x^* = 0$ (loss $= 0$)

What we expect: FTL should learn that oscillating losses cancel out and converge toward $x = 0$.

What actually happens: FTL oscillates between $x_t = +1$ and $x_t = -1$ every round!

- When cumulative gradient $\sum_{s=1}^{t-1} g_s = +1$, FTL chooses $x_t = +1$
- Adversary reveals $g_t = -1$, cumulative becomes 0, FTL flips to $x_{t+1} = -1$
- Adversary reveals $g_{t+1} = +1$, cumulative becomes $+1$, cycle repeats
- Total loss: ≈ 99 (lose 1 on almost every round)
- Regret: $99 - 0 = 99$ (linear in T!)

⚠️ Crisis

The Instability Crisis: Linear Regret from Oscillations

Expected: "Cumulative loss smooths out → converge to $x \approx 0$ → low regret"

Reality: Regret $= 99$ (linear in T, not sublinear!)

Root cause: FTL has **no memory** or **inertia**. It's purely reactive to cumulative data. When losses nearly cancel, tiny perturbations cause huge decision changes. The algorithm is **maximally sensitive** to the most recent loss.

Key insight: FTL has **no memory** or **inertia**. It's purely reactive to cumulative data. When losses nearly cancel, tiny perturbations cause huge decision changes. The algorithm is **maximally sensitive** to the most recent loss.

Comparison: How Bad Is This?

Let's compare FTL to other approaches on the same alternating loss sequence:

📊 **Comparison**

Performance on alternating losses over $T = 100$ rounds:

Algorithm	Cumulative Loss	Regret	Regret Rate
Best fixed decision ($x^* = 0$)	0	0	—
Random decision (baseline)	50	50	$\Theta(T)$
FTL (Follow-The-Leader)	**99**	**99**	$\boldsymbol{\Theta(T)}$
OGD (Online Gradient Descent)	14.1	14.1	$O(\sqrt{T})$

Key observation: FTL performs **worse than random**! The oscillations are so bad that flipping a coin would be better. No amount of problem structure (smoother losses, smaller domains, or strongly convex losses) fixes this—FTL fundamentally cannot handle adversarial noise.

⚠️ **Crisis**

The Fundamental Problem: FTL Cannot Handle Adversarial Noise

Root cause: FTL's decision x_t changes by $O(1)$ when a single loss f_t is added to the cumulative sum. This is **hypersensitivity**.

Mathematical view:

$$x_t = \operatorname*{argmin}_{x \in K} \sum_{s=1}^{t-1} f_s(x) \tag{26}$$

A single new loss f_t with gradient g_t causes:

$$x_{t+1} = \operatorname*{argmin}_{x \in K} \left[\sum_{s=1}^{t-1} f_s(x) \right] + f_t(x) \tag{27}$$

The new term $f_t(x)$ has equal weight to the entire history. No dampening!

What we need: A way to make FTL **stable**—changes in cumulative loss should cause **small** changes in decisions.

The Aha Moment: Add Regularization for Stability

How do we prevent oscillations? We need **inertia**—a force that resists sudden changes in decisions.

💡 Aha Moment

The key insight: Add a **regularizer** $R(x)$ to the cumulative loss. This penalizes decisions that are far from some reference point (usually the origin).

Modified optimization:

$$x_t = \operatorname*{argmin}_{x \in K} \left[\sum_{s=1}^{t-1} f_s(x) \right] + \frac{1}{\eta} R(x) \tag{28}$$

Intuition:

- The term $\sum_{s=1}^{t-1} f_s(x)$ pulls toward minimizing past losses
- The term $\frac{1}{\eta} R(x)$ pulls toward a stable reference (e.g., origin)
- The balance is controlled by η (learning rate)
- Larger η = more weight on losses, less on regularizer (more reactive)
- Smaller η = more weight on regularizer, less on losses (more stable)

Why this helps: Regularization adds **curvature** to the objective. Even when $\sum_{s=1}^{t-1} f_s(x)$ is flat or oscillating, $R(x)$ keeps x_t near a stable point.

This is called Follow-The-Regularized-Leader (FTRL).

Definition: Follow-The-Regularized-Leader (FTRL)

Parameters:

- Decision space: Convex set $K \subseteq \mathbb{R}^d$
- Regularizer: Convex function $R : K \to \mathbb{R}$ with $R(0) = 0$
- Learning rate: $\eta > 0$

Initialization: Choose $x_1 = \operatorname{argmin}_{x \in K} R(x)$ (usually $x_1 = 0$)

For each round t = 1, 2, ..., T:

1. **Decide:** Choose

$$x_t = \operatorname*{argmin}_{x \in K} \left[\eta \sum_{s=1}^{t-1} f_s(x) + R(x) \right] \tag{29}$$

2. **Observe:** Loss function $f_t : K \to \mathbb{R}$ and incur loss $f_t(x_t)$

Common regularizers:

- Euclidean: $R(x) = \frac{1}{2}\|x\|^2$ (for bounded domains)
- Entropic: $R(x) = \sum_{i=1}^{d} x_i \log(x_i)$ (for probability simplex)
- ℓ^1: $R(x) = \|x\|_1$ (for sparse decisions)

Numerical Success: FTRL Achieves $O\left(\sqrt{T}\right)$ Regret

Let's test FTRL on the same adversarial sequence that destroyed FTL.

Setup: Same as before

- $K = [-1, 1]$, alternating $g_t \in \{+1, -1\}$, $T = 100$
- Regularizer: $R(x) = \frac{1}{2}x^2$
- Learning rate: $\eta = 0.1$

FTRL's optimization at each round:

$$x_t = \operatorname*{argmin}_{x \in [-1,1]} \left[0.1 \sum_{s=1}^{t-1} g_s x + \frac{1}{2}x^2 \right] \tag{30}$$

Solution: Take derivative and set to zero:

$$x_t = -0.1 \sum_{s=1}^{t-1} g_s \tag{31}$$

Pattern: x_t oscillates between 0 and -0.1, not between -1 and $+1$!

Comparison at different time horizons:

Comparison

T (rounds)	FTL Regret	FTRL Regret	Ratio	FTRL grows as
100	99	5.0	20×	$\sqrt{T}$
1,000	999	15.8	63×	$\approx 5\sqrt{T}$
10,000	9,999	50.0	200×	$\approx 5\sqrt{T}$
100,000	99,999	158.1	632×	$\approx 5\sqrt{T}$
1,000,000	999,999	500.0	2000×	$\approx 5\sqrt{T}$

Observations:

- FTL: Regret $\approx T - 1 = \Theta(T)$ (linear growth)
- FTRL: Regret $\approx 5\sqrt{T}$ (sublinear growth!)
- Gap grows as $\sqrt{T}$: At $T = 10^6$, FTRL is 2000× better
- Smaller η = more stability = smaller regret (FTRL with $\eta = 0.05$ achieves regret $\approx$ 2.6 at $T = 100$)

Why $\sqrt{T}$? The FTRL regret bound is $\text{Regret}_T \leq \frac{R(x^*)}{\eta} + \frac{\eta G^2 T}{2}$. Balancing initialization cost vs gradient noise with $\eta = \sqrt{\frac{2R(x^*)}{G^2 T}}$ yields $O\left(\sqrt{T}\right)$ regret—optimal for adversarial OCO.

Formal Regret Bound for FTRL

Theorem: FTRL Regret Bound

Let $K \subseteq \mathbb{R}^d$ be a convex set, and let $f_1, ..., f_T : K \to \mathbb{R}$ be convex loss functions with $\|\nabla f_t(x)\| \leq G$ for all t and $x \in K$.

Let $R : K \to \mathbb{R}$ be a 1-strongly convex regularizer with $R(0) = 0$.

Then FTRL with learning rate $\eta > 0$ satisfies:

$$\sum_{t=1}^{T} f_t(x_t) - \min_{x^* \in K} \sum_{t=1}^{T} f_t(x^*) \leq \frac{R(x^*)}{\eta} + \frac{\eta G^2 T}{2} \tag{32}$$

Optimal choice: Setting $\eta = \sqrt{\frac{2R(x^*)}{G^2 T}}$ gives:

$$\text{Regret}_T \leq \sqrt{2R(x^*)G^2 T} = O\left(\sqrt{T}\right) \tag{33}$$

Proof idea: (Full proof in lecture)

The key is to relate FTRL's decisions to a **stability** property:

1. **One-step regret decomposition:** For each round,

$$f_t(x_t) - f_t(x^*) \leq \langle \nabla f_t(x_t), x_t - x^* \rangle \tag{34}$$

by convexity.

2. **Telescoping:** The FTRL updates can be written as:

$$x_{t+1} = \operatorname*{argmin}_{x \in K} \left[\eta \sum_{s=1}^{t} \langle \nabla f_s(x_s), x \rangle + R(x) \right] \tag{35}$$

The gradient sum can be **telescoped** using the Bregman divergence:

$$D_R(x, y) = R(x) - R(y) - \langle \nabla R(y), x - y \rangle \tag{36}$$

3. **Key inequality:** Strong convexity of R gives:

$$D_R(x^*, x_t) - D_R(x^*, x_{t+1}) \geq \eta \langle \nabla f_t(x_t), x_t - x^* \rangle - \frac{\eta^2 G^2}{2} \tag{37}$$

4. **Sum over t:** Summing from $t = 1$ to T and using $D_R(x^*, x_1) = R(x^*)$ and $D_R(x^*, x_{T+1}) \geq 0$:

$$\sum_{t=1}^{T} \langle \nabla f_t(x_t), x_t - x^* \rangle \leq \frac{R(x^*)}{\eta} + \frac{\eta G^2 T}{2} \tag{38}$$

5. **Apply convexity:** Use $f_t(x_t) - f_t(x^*) \leq \langle \nabla f_t(x_t), x_t - x^* \rangle$ to get the regret bound.

Why this bound is tight: The $\sqrt{T}$ rate is **unavoidable** in adversarial online learning. Lower bounds show that any algorithm must have $\text{Regret}_T = \Omega\left(\sqrt{T}\right)$ in the worst case.

Connection to Online Gradient Descent

FTRL and OGD look different, but they're deeply connected.

Comparison

FTRL vs OGD:

Aspect	FTRL	OGD
Update rule	$x_t = \text{argmin}_{x \in K} \left[\eta \sum_{s=1}^{t-1} f_s(x) + R(x)\right]$	$x_{t+1} = \Pi_K(x_t - \eta \nabla f_t(x_t))$
Perspective	Global: minimize cumulative loss + regularizer	Local: take gradient step + project
Regularizer	Explicit $R(x)$ in objective	Implicit (related to projection)
Regret bound	$\frac{R(x^*)}{\eta} + \frac{\eta G^2 T}{2}$	$\frac{D^2}{2\eta} + \frac{\eta G^2 T}{2}$
Optimal η	$\sqrt{\frac{2R(x^*)}{G^2 T}}$	$\frac{D}{G\sqrt{T}}$

Key insight: When $R(x) = \frac{1}{2}\|x\|^2$ and K is bounded with diameter D, FTRL and OGD give essentially the same regret: $O\left(DG\sqrt{T}\right)$.

Difference in practice:

- FTRL: Better when $R(x)$ matches problem structure (e.g., entropic regularization for simplex)
- OGD: Simpler to implement (just gradient + projection)

Remarkable fact: For Euclidean regularization $R(x) = \frac{1}{2}\|x\|^2$, FTRL and OGD produce **exactly the same decisions** x_t! They're two views of the same algorithm.

Summary: The Power of Regularization

Key Takeaway

What we learned:

1. **FTL fails badly:** Without regularization, Follow-The-Leader achieves $\Theta(T)$ regret on adversarial losses due to oscillations.
2. **FTRL stabilizes:** Adding regularizer $R(x)$ dampens oscillations, reducing regret from $\Theta(T)$ to $O\left(\sqrt{T}\right)$.
3. **Optimal tradeoff:** Learning rate η balances initialization cost ($\frac{R(x^*)}{\eta}$) vs accumulated noise ($\frac{\eta G^2 T}{2}$).
4. **Universal rate:** The $\sqrt{T}$ regret rate is optimal for adversarial online learning—no algorithm can do better in the worst case.
5. **Connection to OGD:** FTRL and OGD are equivalent for Euclidean regularization, showing deep unity in online learning algorithms.

The big picture: Regularization provides **stability** against adversarial noise—a fundamental algorithmic principle: **smooth algorithms are robust algorithms**.

Strong Convexity: From $\sqrt{T}$ to $\log T$ Regret

We've achieved $O\left(\sqrt{T}\right)$ regret with FTRL using convex regularizers. Can we do better with additional structure?

The Natural Question: Can Any Convex Regularizer Work?

We've been using $R(x) = \frac{1}{2}\|x\|^2$, which is convex and smooth. What about other choices?

Candidates:

- $R(x) = \|x\|$ (L1 norm—promotes sparsity)
- $R(x) = \|x\|^{\frac{3}{2}}$ (superlinear growth)
- $R(x) = \|x\|^2$ (our current choice)

All are convex. Do they give the same regret bound?

Numerical Exploration: Testing Weak Regularizers

Let's try a **weakly convex** regularizer: $R(x) = |x|$ (absolute value) on $K = [-1, 1]$.

Test 1: Alternating gradients

- Setup: $K = [-1, 1]$, $R(x) = |x|$, $\eta = 0.1$, alternating $g_t \in \{+1, -1\}$
- FTRL update: $x_t = \text{argmin}_{x\in[-1,1]}\left[0.1\sum_{s=1}^{t-1} g_s x + |x|\right]$
- Result: $x_t = 0$ for all t (zero regret, looks perfect!)
- Why: Objective minimum at $x = 0$ when cumulative gradient $|0.1\sum g_s| < 1$

Test 2: Persistent gradients

- Setup: Same regularizer, but all $g_t = +1$ (consistent pressure toward $x = -1$)
- Best fixed decision: $x^* = -1$
- What happens: FTRL stays at $x_t = 0$ until $\sum_{s=1}^{t-1} g_s > 10$ (dead zone!)
- Regret at $T = 100$: approximately 145 (vs expected $\sqrt{100} = 10$)

⚠ Crisis

The Dead Zone Problem: Weak Regularizers Cannot Adapt

Comparison:

- L1 regularizer ($R(x) = |x|$): Regret ≈ 145 (14× worse than expected)
- L2 regularizer ($R(x) = \frac{1}{2}x^2$): Regret ≈ 10 (as expected for $O\left(\sqrt{T}\right)$)

Root cause: $R(x) = |x|$ has **zero curvature** at origin—subgradient is a set $[-1, +1]$, not unique. This creates a "dead zone" where decisions don't change until cumulative gradient crosses threshold.

Key insight: Not all convex regularizers are equal. We need **strong convexity**—a quantitative lower bound on curvature.

The Aha Moment: Strong Convexity

Aha Moment

The key insight: We need **strong convexity**—a quantitative lower bound on curvature.

Definition: A function $R : K \to \mathbb{R}$ is **α-strongly convex** if:

$$R(y) \geq R(x) + \langle \nabla R(x), y - x \rangle + \frac{\alpha}{2}\|y - x\|^2 \tag{39}$$

for all $x, y \in K$, where $\alpha > 0$ is the **strong convexity parameter**.

Intuition:

- Regular convexity: $R(y) \geq R(x) + \text{linear term}$ (no curvature requirement)
- Strong convexity: $R(y) \geq R(x) + \text{linear term} + \frac{\alpha}{2}\|y - x\|^2$ (quadratic penalty!)

Why this helps: The quadratic term $\frac{\alpha}{2}\|y - x\|^2$ ensures that moving away from the current point x incurs a penalty proportional to the **squared distance**. This prevents dead zones and ensures smooth adaptation.

Examples:

- $R(x) = \frac{1}{2}\|x\|^2$: 1-strongly convex (take $\alpha = 1$)
- $R(x) = |x|$: 0-strongly convex (not strongly convex, $\alpha = 0$)
- $R(x) = \sum_{i=1}^{d} x_i \log(x_i)$ (negative entropy): α-strongly convex w.r.t. ℓ^1 norm

Key property: Strong convexity with $\alpha > 0$ ensures unique gradients (no subgradient sets) and smooth optimization landscapes.

How Strong Convexity Improves Regret

Let's revisit the FTRL regret bound with strong convexity.

Theorem: FTRL with Strong Convexity

Let $R : K \to \mathbb{R}$ be **α-strongly convex** with $R(0) = 0$.

Then FTRL with learning rate $\eta > 0$ satisfies:

$$\text{Regret}_T \leq \frac{R(x^*)}{\eta} + \frac{\eta G^2 T}{2\alpha} \tag{40}$$

Key difference: The second term has $\frac{1}{\alpha}$ instead of 1!

Optimal choice: Setting $\eta = \sqrt{\frac{2\alpha R(x^*)}{G^2 T}}$ gives:

$$\text{Regret}_T \leq \sqrt{\frac{2R(x^*)G^2 T}{\alpha}} = O\left(\sqrt{T}\right) \tag{41}$$

But we can do better! With **time-varying** learning rates η_t, we can achieve:

$$\text{Regret}_T = O\left(\frac{G^2}{\alpha} \log T\right) \tag{42}$$

This is a **qualitative improvement**: logarithmic vs square root growth!

Why does strong convexity help?

The proof (omitted here, see advanced notes) uses the fact that strong convexity controls the **stability** of FTRL's decisions. Specifically:

1. **Small gradients → small movements:** When $\|\nabla f_t(x_t)\| = g$ is small, the change $\|x_{t+1} - x_t\|$ is $O(\frac{g}{\alpha})$.
2. **Noise accumulation is bounded:** Over T rounds, the cumulative effect of gradient noise is:

$$\sum_{t=1}^{T} \|x_{t+1} - x_t\|^2 = O\left(\frac{G^2 T}{\alpha^2}\right) \tag{43}$$

3. **Logarithmic regret with decreasing η_t:** Using $\eta_t = \frac{1}{\alpha t}$, the regret becomes:

$$\text{Regret}_T = O\left(\frac{G^2}{\alpha} \sum_{t=1}^{T} \frac{1}{t}\right) = O\left(\frac{G^2}{\alpha} \log T\right) \tag{44}$$

The $\log T$ comes from the harmonic sum $\sum_{t=1}^{T} \frac{1}{t} = O(\log T)$.

Numerical Comparison: $\sqrt{T}$ vs $\log T$ Growth

Let's compare regret growth for regular convex vs strongly convex regularizers.

Setup:

- Regular convex: $R(x) = |x|$ with $\alpha = 0$
- Strongly convex: $R(x) = \frac{1}{2}x^2$ with $\alpha = 1$
- Losses: Mix of adversarial sequences
- Optimal learning rates for each

Comparison

Regret comparison at different time horizons:

T (rounds)	Regular Convex $O(\sqrt{T})$	Strongly Convex $O(\log T)$	Ratio (improvement)	Gap
100	50	9.2	5.4×	41
1,000	158	13.8	11.4×	144
10,000	500	18.4	27.2×	482
100,000	1581	23.0	68.7×	1558
1,000,000	5000	27.6	181×	4972

Observations:

1. **Initial advantage:** At $T = 100$, strongly convex is 5× better
2. **Growing gap:** At $T = 10^6$, strongly convex is 181× better
3. **Unbounded improvement:** As $T \to \infty$, ratio $\frac{\sqrt{T}}{\log T} \to \infty$

The key insight: Strong convexity doesn't just improve constants—it changes the **asymptotic rate**. This is a **qualitative** improvement, not quantitative.

Visualizing the Difference

At $T = 1,000,000$:

- $\sqrt{T} \approx 1000$ (grows like 3 digits per 6 orders of magnitude)
- $\log T \approx 14$ (grows like 1 digit per 10× increase)

Practical impact: In million-round experiments (common in online advertising, real-time bidding, etc.), logarithmic regret means near-constant performance after initial learning. Square-root regret means perpetual growth.

Key Takeaway

Why strong convexity gives $\log T$ regret:

1. **Quadratic penalty:** $\frac{\alpha}{2}\|y - x\|^2$ prevents dead zones and ensures smooth adaptation.
2. **Controlled noise:** Gradient noise accumulation is bounded by $\frac{G^2}{\alpha^2}$, not G^2.
3. **Time-varying learning rates:** With $\eta_t = \frac{1}{\alpha t}$, the regret decomposes as:

$$\text{Regret}_T = O\left(\sum_{t=1}^{T} \frac{G^2}{\alpha t}\right) = O\left(\frac{G^2}{\alpha} \log T\right) \tag{45}$$

4. **Qualitative improvement:** $\log T$ grows **much slower** than $\sqrt{T}$. At $T = 10^6$, this is 14 vs 1000 (70× improvement).

The big picture: Strong convexity transforms online learning from **perpetual adaptation** ($\sqrt{T}$ regret keeps growing) to **near-convergence** ($\log T$ regret flattens out). This is the difference between "learning slowly forever" and "learning quickly then stabilizing."

Practical Regularizers with Strong Convexity

Which regularizers are strongly convex?

📊 **Comparison**

Common regularizers and their strong convexity:

Regularizer $R(x)$	Domain K	α (strong cvx)	When to Use
$\frac{1}{2}\|x\|^2$	Any bounded	$\alpha = 1$	General purpose, Euclidean geometry
$\sum_{i=1}^{d} x_i \log(x_i)$ (neg. entropy)	Probability simplex	$\alpha = 1$ (ℓ^1 norm)	Expert advice, portfolio optimization
$\frac{\lambda}{2}\|x\|^2 + \frac{1-\lambda}{2}\|x\|_1^2$	Any bounded	$\alpha = \min(\lambda, 1-\lambda)$	Sparse + smooth tradeoff
$\|x\|$ (L1 norm)	Any	$\alpha = 0$	❌ Not strongly convex, avoid for FTRL
$\|x\|^p$ for $p \in (1,2)$	Any	$\alpha = 0$	❌ Not strongly convex

Key guideline: For online learning, always use strongly convex regularizers (quadratic or entropic). Avoid L1 and subquadratic norms.

Connection to Coordinate Descent and AdaBoost

Strong convexity appears throughout machine learning theory. Two important connections:

1. **AdaBoost as FTRL:** AdaBoost (Week 6) can be viewed as FTRL with:
   - Decision space: Probability distributions over weak learners
   - Regularizer: Negative entropy $R(p) = \sum_i p_i \log(p_i)$
   - Strong convexity: $\alpha = 1$ w.r.t. ℓ^1 norm

   This explains why AdaBoost achieves exponentially decaying training error: strong convexity gives $\log T$ regret, which translates to exponentially small error on training data.

2. **Coordinate descent:** In batch optimization, strongly convex objectives ensure:
   - Unique minimizers
   - Linear convergence rates (exponential decrease in error)
   - Stability under perturbations

Strong convexity is the **key structural property** that enables fast convergence in both online and batch settings.

When Strong Convexity Fails: The Necessity of Curvature

Can we get $\log T$ regret without strong convexity?

⚠️ **Crisis**

Lower bound: $\sqrt{T}$ is necessary without strong convexity

Theorem (informal): For any online learning algorithm using a regularizer with $\alpha = 0$ (not strongly convex), there exists an adversarial sequence such that:

$$\text{Regret}_T = \Omega\left(\sqrt{T}\right) \tag{46}$$

Intuition: Without quadratic penalty, adversary can force "flat regions" where the algorithm cannot distinguish between good and bad decisions until $O\left(\sqrt{T}\right)$ loss has accumulated.

Conclusion: Strong convexity ($\alpha > 0$) is **necessary** for $\log T$ regret, not just sufficient.

Practical takeaway: Always check if your regularizer is strongly convex. If $\alpha = 0$, you're limited to $\sqrt{T}$ regret. If $\alpha > 0$, you can achieve $\log T$ with proper learning rate schedules.

Summary: The Power of Curvature

🎯 **Key Takeaway**

What we learned:

1. **Not all convex regularizers are equal:** $R(x) = |x|$ gives poor performance due to lack of curvature ($\alpha = 0$).
2. **Strong convexity is crucial:** α-strongly convex regularizers with $\alpha > 0$ enable $O(\log T)$ regret vs $O\left(\sqrt{T}\right)$.
3. **Qualitative improvement:** At $T = 10^6$, strongly convex regularizers are $181\times$ better—and the gap grows unboundedly.
4. **Practical regularizers:** Use $\frac{1}{2}\|x\|^2$ (Euclidean) or negative entropy (for simplex). Avoid L1 and subquadratic norms.
5. **Connection to AdaBoost:** Negative entropy's strong convexity explains AdaBoost's exponential convergence on training data.

The big picture: Strong convexity is the key property that separates "slow learning" ($\sqrt{T}$) from "fast learning" ($\log T$) in online optimization. This principle extends to batch methods (coordinate descent) and boosting (AdaBoost). Curvature enables convergence.

Summary and Looking Ahead

We began with a question: can we make good predictions when the world is adversarial? The answer is yes, and the journey revealed deep connections across learning theory.

What we've accomplished:

- **OCO framework:** Continuous decisions via gradients (not discretization)
- **OGD algorithm:** $O\left(\sqrt{T}\right)$ regret using gradient descent + projection
- **FTRL framework:** Regularization provides stability against adversarial noise
- **Strong convexity:** Improves $O\left(\sqrt{T}\right)$ to $O(\log T)$ regret—a qualitative leap

Key techniques:

- Convexity transforms adversarial unpredictability into geometric control
- Regularization dampens oscillations (FTL fails with $\Theta(T)$ regret)
- Multiplicative weight updates unify discrete (MWU) and continuous (FTRL with entropy) cases
- Strong convexity enables logarithmic regret via time-varying learning rates

Looking Ahead: Week 11-12

Next we'll study **deep learning theory**—neural network expressivity, training dynamics, and generalization. Connection: SGD for training neural networks is online gradient descent! The theory we developed this week (OGD, implicit regularization) provides foundations for understanding deep learning optimization.

→ Week 11 Preview: Why Deep Learning Works

Week 11-12 tackle the mystery: Why do overparameterized neural networks generalize? The answer involves:

1. **Optimization:** SGD (= OGD on mini-batches) finds good minima via implicit regularization
2. **Implicit bias:** The $O\left(\sqrt{T}\right)$ convergence analysis extends to show SGD prefers "simple" solutions
3. **NTK regime:** In infinite width, networks behave like kernel methods—and we can analyze them!

Everything we learned about OGD, regularization, and convergence rates directly applies.

🎯 Key Takeaway

Online learning provides a robust framework for sequential decision-making without distributional assumptions. The Multiplicative Weights Update method and its variants (OGD, FTRL) achieve optimal $O\left(\sqrt{T}\right)$ regret through simple multiplicative updates. These algorithms have applications ranging from game theory to optimization to modern machine learning, and provide theoretical foundations for understanding SGD and adaptive optimizers in deep learning.

Final backbone: The exponential reweighting principle—$w \leftarrow we^{-\eta\ell}$ or equivalently $w \leftarrow w(1-\eta\ell)$—is perhaps the most important algorithmic idea in online learning. It appears in expert advice (MWU), boosting (AdaBoost), game theory (fictitious play), convex optimization (OGD/FTRL), and deep learning (SGD/Adam). Recognizing this pattern across domains is key to understanding modern machine learning.

📊 Confidence Check

Final Self-Assessment: Before moving to Week 11, rate your mastery (1-5):

- □ I can explain the online learning framework and how it differs from PAC learning
- □ I understand why regret $R_T = o(T)$ is the right goal
- □ I can derive the Weighted Majority bound using potential functions
- □ I understand why randomization improves from factor 2 to factor 1
- □ I can prove the MWU regret bound for real-valued losses
- □ I can analyze OGD using telescoping distance arguments
- □ I understand FTRL and how regularizer choice determines the algorithm
- □ I see the connection between AdaBoost, MWU, OGD, and FTRL
- □ I understand why $O\left(\sqrt{T}\right)$ regret is optimal
- □ I can apply these ideas to new problems (portfolio optimization, game theory, etc.)

If most are 4-5: You're ready for deep learning theory! **If most are 3:** Review the proof templates and work through practice problems **If most are 1-2:** Revisit the mental models and backward/forward connections